

# 引擎接地的调度诊断解释层：LLM 在 FJSP+MAPF 耦合系统中的 定位、归因、可回查证据与可仿真干预

SkyResearch

2026 年 9 月 6 日 v2.3 草稿（四档模型主表含推理行 + hard 视图质检 + CF-INT 差距 + 反事实执行 + unseen 留出两档；agentic 与 frontier 待补）

*When Do LLMs Fail at Engine-Grounded Scheduling Diagnosis?*  
*Engine-Grounded Diagnosis with Executable Counterfactual Verification*

## 摘要

FJSP+MAPF 耦合系统产生丰富的运行工件（指标轨迹、事件流、计划修订账本、异常日志），但缺少把它们变成可理解因果叙述的层。本文定义调度系统**事后诊断**任务  $\mathcal{D}: (\tau, q) \mapsto e$ ——给定运行档案  $\tau$  与查询  $q$ ，产出解释  $e = (\text{定位}, \text{归因}, \text{证据}, \text{干预})$ ，并给出使 LLM 输出可信的三重机制：（1）**注入式真值**——数据由引擎受控注入制造而非采集，标准答案零人工标注；（2）**可回查证据**——档案中每条事实带证据 ID，评分器逐条程序化核对；（3）**可仿真干预**——干预建议在引擎中以同种子反事实重仿真取得  $\Delta KPI$ ，LLM 不被允许“讲故事”。我们构建首个耦合调度诊断基准 cases\_v2：随机化注入网格 + CRN 无故障孪生对照 + 涌现失效配方 + Generator-Executor-Verifier 核验闭环，52 个核验场景产出 104 个 easy/hard 孪生案例；核验器在首次实战中即拦截 26 个缺陷场景（含 13 个孪生暴露的混因配对）。规则基线在 easy/hard 孪生上的定位准确率 59.6%  $\rightarrow$  41.3%、JRA 59.6%  $\rightarrow$  34.6% 的塌陷证实了基准的区分力：无事件耦合失效（活锁/拥塞）构成对 LLM 泛化推断能力的受控测试。与 LLM-CO（HeurAgenix、CO-Bench 的生成侧）与在线决策（ReflecSched）不同，本层是事后诊断且可被引擎证伪。本文结论限于冻结参数、非 agentic、单轮打包设定下的本地模型（Qwen3-4B/8B/32B）；frontier、reasoning 与主动取证模型上的表现为开放问题。

## 1 引言

姊妹篇们把耦合系统当作被优化对象；本篇把它当作被解释对象。动机：车间运维面对的不是一个最优解，而是“为什么这批单晚了/为什么 AGV 都在等”——这需要把

指标轨迹、事件流与计划演变合成因果叙述。全文回答一个研究问题 (RQ1): **LLM 是在诊断调度失效, 还是只是在读显式的故障痕迹?** easy/hard 孪生 (§4) 与无事件涌现失效即为回答此问题而设计。

三个社区现状: (i) LLM-CO 用 LLM 生成启发式或搜算法 (FunSearch、EoH [1]、ReEvo [2]、HeurAgenix [3]、CO-Bench [4]), 属于生成侧, 无诊断视角; (ii) LLM 在线决策读轨迹提经验但服务于下一轮动作 (ReflecSched [5]), 评测其失败的是 DynaSched-Bench [6]——但它诊断的对象是 LLM 调度器本身, 而非调度系统; (iii) AIOps 根因分析 (RCACopilot [7]、mABC [8]、ITBench [9]、Cloud-OpsBench [10]、OpenRCA [11]) 与我们的评测哲学最近——受控故障注入供 ground truth、agentic 工具取证——但其验证止步于人工标注与测试, 无重仿真量化, 也无跨层耦合失效。“LLM 输出的重仿真验证”最近的先例是数字孪生的实施前验证 [12] 与启发式优化的 matched-seeds 重仿真 [13], 二者对象均非诊断结论。

## 2 诊断任务形式化

**定义 1** (轨迹).  $\tau = (\mathbf{m}, \mathbf{e}, \mathbf{r}, \Sigma)$ : 摘要 KPI 表 (20 项聚合指标)、事件样本  $\mathbf{e}$ 、计划修订账本  $\mathbf{r}$ 、配置  $\Sigma$ 。

**定义 2** (诊断).  $\mathcal{D} : (\tau, q) \mapsto e$ , 解释  $e = (\ell_t, \ell_r, h, V, \iota)$ : 定位类型  $\ell_t$  与目标  $\ell_r$  (双字段:  $\ell_t \in \{\text{machine}, \text{agv}, \text{machine\_agv}, \text{stochastic}, \text{task\_pool}, \text{corridor}, \text{machines}, \text{none}\}$ ,  $\ell_r$  如 *machine:3*)、归因  $h$  (8 类受控词表 + *unseen* 逃生舱)、证据  $V$  (档案内带 ID 事实的引用集合)、干预  $\iota$  (引擎可执行配置词表: *padding/fleet/periodic\_revision/assigner*)。

**评测** (ground truth 来自受控注入):

$$\text{定位} = \Pr[\ell_t = \ell_t^* \wedge \ell_r = \ell_r^*] \text{ (仅类型对计 0.5)}, \quad \text{归因} = \Pr[h = h^*] \text{ (disruption 族宽容计 0.5)}, \quad (1)$$

$$\text{JRA} = \Pr[\text{定位满分} \wedge h = h^*], \quad \text{忠实度} = \frac{1}{|V|} \sum_{v \in V} \mathbb{I}[v \text{ 可回查}], \quad (2)$$

干预成功率 =  $\Pr[\Delta \text{KPI}(\iota) \geq \delta]$  (同种子反事实重仿真)。忠实度在 statement 级执行: 除可回查外, 辅以对抗注入 (植入伪证据检验是否转引) 与证据消融 (删除被引证据检验结论是否漂移), 防止 post-rationalization [14]。干预成功率即**诊断效用** (Diagnostic Utility)  $\mathcal{U} = \mathbb{E}[\Delta \text{KPI}(\pi(e))]$  的经验估计——如同医学上的 *diagnosis*  $\rightarrow$  *treatment*  $\rightarrow$  *outcome*: 我们不问 “LLM 说得像不像专家”, 而问 “若按其解释施治, 系统能否被修复”。

## 3 架构

1. **证据打包器**: episode 记录  $\rightarrow$  带证据 ID 的紧凑档案 (KPI:\*/EVT001.../REV:\*/CFG:\*,  $\leq 8k$  token; 剥离热力图)。每条事实可引用、可回查, 是忠实度可程序化核验的前

提。

2. **诊断策略**: 同一数据底座、两种观测视图——(a) 单轮打包: 档案整体入上下文, 一次产出四元组 (消融臂); (b) *agentic* 工具层 (设计中): 引擎暴露为查询工具 (查 KPI/查事件/查修订/提交干预  $\rightarrow$  触发反事实), 主动取证 (主系统)。2026 年的运维基准已明确批评单轮打包为 *passive classification* [10], 双层设计使我们无论何者胜出均有结论可写。输出经 *JsonRegen* 循环收口; 答案期采样聚合。
3. **反事实执行器**: 解析  $\iota \rightarrow$  修改配置  $\rightarrow$  同种子重仿真  $\rightarrow \Delta KPI$ 。采用公共随机数 (CRN) 配对差报告,  $\geq 30$  种子做 Wilcoxon 显著性 [15]; 失败干预记  $\Delta KPI = -\infty$  不丢弃。该判分器不依赖标签——词表外的新失效模式亦可评 (忠实度  $+\Delta KPI + \text{unseen}$  逃生舱)。
4. **训练阶梯** (评测框架的反哺): frozen 提示 A/B  $\rightarrow$  ReEvo 式反思精化 (喂 best-so-far  $\Delta KPI$  对比证据)  $\rightarrow$  反事实 DPO (同案例 “有效/无效干预” 构成偏好对, 可验证奖励, 无需人工标注)。

## 4 cases\_v2: 注入式诊断基准

**Generator–Executor–Verifier 闭环**: 场景生成器从网格抽样 (实例 mk01–03  $\times$  策略  $\times$  车队  $K \in \{2, 3, 4, 6\} \times$  随机种子  $\times$  随机化故障目标/时刻/时长), 跑批器以进程隔离 + 硬超时执行, 核验器按签名谓词判定——不达标即丢弃, 宁可缺数不给错标签。

**场景家族**——按标签的认识论地位二分: 注入式因果失效 (injected causal failures):  $do(F=1)$  vs  $do(F=0)$ , 标签由构造为真, CRN 孪生下是干净的干预效应; 涌现式系统失效 (emergent systemic failures): 饥饿活锁、走廊拥塞等纯 FJSP/纯 MAPF 中不存在的耦合病理——没有任何显式故障事件, 所有组件各自 “正常” 而组合死亡, 标签由签名谓词定义 (现象学诊断而非  $do()$  意义上的根因), 只有核验器确认签名后才入库。具体配方: (i) 注入故障 36 (机器 16/双故障 12/随机扰动 8); (ii) 基线 6; (iii) 涌现配方 8; (iv) **CRN 无故障孪生** 28: 每个注入场景配同配置无故障孪生, 孪生与故障案的 makespan 差即干预应恢复量; 孪生未完工则配对作废。cases\_v2 是该过程式生成器的一个抽样实例——基准以生成器的覆盖域与 held-out 泛化定义, 规模可持续扩张。

核验器首次实战拦截 26 场景: 13 个注入对因孪生未完工作废 (greedy 臂在迷宫图系统性活锁——若无 CRN 孪生, 这批案例将带混因标签入库)、5 个涌现配方签名未复现、1 对触发引擎接口屏障死循环 (300s 硬超时拦截)。

## 5 实验: 规则基线的 easy/hard 塌陷

规则基线 (事件优先 + 阈值签名, v1 逻辑适配双字段) 在 104 案例上:

| 归因类别                   | 案例数 (easy+hard) | 来源                       |
|------------------------|-----------------|--------------------------|
| baseline               | 34              | 基线 + 孪生                  |
| blocking_livelock      | 30              | 基线改标 (greedy 迷宫活锁, 签名确认) |
| disruption_machine     | 14              | 随机注入                     |
| disruption_machine_agv | 10              | 随机双故障                    |
| starvation_livelock    | 8               | 涌现配方 (签名确认)              |
| disruption_stochastic  | 8               | preset (完工者保留)           |

表 1: cases\_v2 构成: 52 个核验场景  $\times$  easy/hard 孪生 = 104 案例。easy 含事件流 (考阅读), hard 删除事件流只留统计特征 (考推断)。

| 变体          | 定位    | 归因 (严格) | JRA   | 解析率  |
|-------------|-------|---------|-------|------|
| easy (含事件流) | 0.596 | 0.596   | 0.596 | 1.00 |
| hard (无事件流) | 0.413 | 0.481   | 0.346 | 1.00 |
| 首异常 (easy)  | 0.462 | 0.462   | 0.462 | 0.31 |
| 首异常 (hard)  | 0.327 | 0.327   | 0.327 | 0.00 |

表 2: 规则基线与首异常启发式 (G1.2 strawman) 的 easy/hard 对照。对比 v1 试点“95%/97.5%”的虚胖 (答案位于事件字段首行、故障目标恒为 0 号机), 随机化与无事件耦合失效构成实质难度。首异常启发式取最早事件实体作答: easy 档即低于规则 (“首个异常  $\neq$  根因”: 涌现类无事件可考、随机扰动类归因缺位); 计划注入主事件之外的诱饵事件时间陷阱率为 0/12 (H5 预注册的诚实负结果); hard 档无事件流时全盲, 塌陷至 0.327。规则基线方向由一次独立实现交叉验证 (另一代码线 24 案例 greedy 单臂试点 easy $\rightarrow$ hard 定位 0.812 $\rightarrow$ 0.625, 与主表同向)。

塌陷幅度的含义: 规则只能回答预埋的签名, 无事件耦合失效上目标不可知 (部分分 0.5 封顶)。反事实推理文献预言 LLM 在 abduction 型任务上系统性弱 [16, 17]; 本基准给出首个调度域实证——结果如下。

全量结果给出五个发现: (F1) 非思考档内 hard 档 JRA 随规模 **单调降至零** (4B 0.038  $\rightarrow$  8B 0.019  $\rightarrow$  32B 0.000), easy 档非单调 (0.212  $\rightarrow$  0.038  $\rightarrow$  0.154) 且三档全部远低于规则的 0.596; 开启思考链的 30B MoE 是唯一在 hard 档取得两位数的 LLM (JRA 0.115, 为非思考档最高 0.038 的三倍) ——**思考链部分挽回溯因损失, 但仍低于规则基线的 0.346**; (F2) hard 档崩塌至接近零——无事件 abductive 推断的缺口在调度域被受控证实 (hard 视图三防线质检 52/52 通过, 塌陷非视图缺陷所致, 见 qc\_hard\_report.json); (F3) **忠实—正确分离**: 四档规模的忠实度均高达 0.95–1.00 且与正确率无关 (8B hard 定位全零、32B JRA 0.077 仍句句可回查) ——“引用成立  $\neq$  诊断成立”的实测证据, 忠实度必须独立成指标; (F4) **基率忽视**: 4B 把 30/34 个无故障基线诊断为饥饿/拥塞/瓶颈, 8B 更甚 (29/33 判为 blocking), 32B 达到 34/34 全部误

| 诊断器                      | 变体   | 定位    | 归因 (严格) | JRA   | 忠实度   |
|--------------------------|------|-------|---------|-------|-------|
| 规则                       | easy | 0.596 | 0.596   | 0.596 | –     |
| 规则                       | hard | 0.413 | 0.481   | 0.346 | –     |
| Qwen3-4B (frozen)        | easy | 0.250 | 0.481   | 0.212 | 0.969 |
| Qwen3-4B (frozen)        | hard | 0.087 | 0.308   | 0.038 | 0.990 |
| Qwen3-8B (frozen)        | easy | 0.115 | 0.269   | 0.038 | 1.000 |
| Qwen3-8B (frozen)        | hard | 0.000 | 0.269   | 0.019 | 0.990 |
| Qwen3-32B (frozen)       | easy | 0.154 | 0.154   | 0.154 | 1.000 |
| Qwen3-32B (frozen)       | hard | 0.067 | 0.000   | 0.000 | 1.000 |
| Qwen3-30B-A3B (thinking) | easy | 0.308 | 0.404   | 0.308 | 0.979 |
| Qwen3-30B-A3B (thinking) | hard | 0.173 | 0.327   | 0.115 | 0.949 |

表 3: cases\_v2 主对照表 (104 案例, 温度 0, 干净档案视图)。4B/8B 经 Ollama 本地端点 (思考链无法关闭, 推理耗 3–5k token); 32B 经 vLLM 关闭思考链; 30B-A3B-Thinking 为推理模型 (思考链开启, reasoning parser 分离)。

诊, 30B-A3B-Thinking 同样 34/34 (28 判 blocking、6 判 starvation) ——四档模型都没有“正常运行包络”概念, 且不随规模与推理能力消失; (F5) **词表锚定失败**: 60 个走廊拥塞案例中 loc\_type=“corridor”从未出现, 但 cause 判断部分正确 (4B 16/30、8B 14/30) ——知道病类、说不出病位; 32B 进一步出现**词表坍塌**: 88/104 收敛到单一标签 machine\_bottleneck, 并产出词表外标签 disruption\_agv (2 例)。干预药房约束四档模型均 100% 合规 (104 案例全部落在引擎可执行词表内; 30B-A3B-Thinking 90/104 选择同一干预 padding, 坍塌亦见于处方侧)。frontier 模型与 agentic 取证能否扭转 F1/F2 是本基准的开放问题。[LLM\_SCALE\_TABLE]

**范围与局限**: 本节结论的范围是冻结参数、单轮打包的本地 LLM (Qwen3 家族 4B/8B/32B 及 30B-A3B-Thinking 推理档)。三点须知: (i) 4B/8B 经 Ollama 端点、思考链无法关闭 (推理烧 3–5k token 后仍提取出最终答案); 32B 经 vLLM 以 enable\_thinking=false 关闭思考链——规模轴上存在这一服务栈差异, 解释 32B easy 档相对 8B 的非单调回升时应计入; (ii) 全部为单轮打包诊断 (整个档案一次给出), 非 agentic 主动取证; (iii) 据此, 本文标题采取 “When Do LLMs Fail” 的条件式发问而非 “LLM 不能诊断” 的全称否定——frozen 小/中模型在本基准上的失败已实测, reasoning 与 agentic 是否翻案待 §5.2 的工具层与 frontier 行回答。

## 5.1 CF-INT: 塌陷的信息归因

为区分 hard 档塌陷是“信息不足”还是“溯因无能”, 为 16 个注入场景生成 INT 视图 (hard + **公开注入参数**, 即 v0.2 防线 3 的兜底归因)。Qwen3-32B 在 INT 档整体 JRA 回升至 0.75 (hard: 0.000), 分解如下:

| 归因类                    | hard JRA | INT JRA | 差距   |
|------------------------|----------|---------|------|
| disruption_machine     | 0.5      | 1.0     | +0.5 |
| disruption_machine_agv | 0.1      | 1.0     | +0.9 |
| disruption_stochastic  | 0.0      | 0.0     | 0    |

表 4: CF-INT 差距 (Qwen3-32B, 注入类)。

两点结论: (i) 防线 2 标记的混淆对 (machine\_agv  $\times$  stochastic, hard 视图标记轮廓重合 1.0) 被公开注入参数**完全消解** (0.1 $\rightarrow$ 1.0) ——坍塌是**信息不对称**而非模型无能, 与 F2/F3 共同构成“观测形态决定诊断上界”的证据链; (ii) preset 型故障 (stochastic) INT 仍为 0——其公开参数只有 preset 名与种子, 无具体时刻/目标可公开, INT 消歧对概率型故障天然失效 (该类需要公开注入事件时间戳, 留作 v2.2)。

## 5.2 反事实执行与 unseen 留出

**反事实执行器** (药房词表  $\rightarrow$  配置增量  $\rightarrow$  同 seed 重仿真  $\rightarrow$   $\Delta$ KPI, 对齐 CRN 孪生的应恢复量) 在规则臂 9 例上: 6 例有效 (成功率 0.667; 最佳 padding/fleet  $\Delta$ KPI +99/+375), 1 例有害 ( $\Delta$ KPI -67) ——“无效干预率”开始区分“敢开方”与“开对方”。同 seed 重仿真本身获得**逐位复现实证** (重跑 makespan 与入库值一致), CRN 判分的确定性假设成立。

**unseen 留出** (leave-one-class-out, Patch B): 先后把 starvation\_livelock 与 blocking\_livelock 从归因词表抽走, Qwen3-32B 两档轮换完全一致——诚实率 **0.0**、沉默失败率 **1.0**、假阳性率 **0.0**: 模型从不使用 unseen 逃生舱承认不认识 (词表坍缩支配一切), 但也不滥用逃生舱。其余类别轮换与干预有效性维度进行中。

**测量有效性**: 开发过程曾因 ground\_truth 随档案整体进入 prompt 而得到 100% 的虚假满分 (该全量结果作废留档)。此后诊断器只允许经 model\_view() 白名单取档案本体, 并确立纪律: 异常好的分数先疑测量、再疑模型。该事故与处置作为基准工程的一部分一并公开。

## 6 结论

本文定义了引擎接地的调度诊断任务 (双字段定位/归因/证据/干预 + 三层判分 + 反事实验证), 交付 Generator-Executor-Verifier 式基准 cases\_v2 (104 案例, 核验闭环拦截 26 缺陷场景) 与首个 easy/hard 坍塌基线。四档 LLM (含推理档) 全量评测、反事实执行器与 unseen 留出已完成; agentic 工具层、KPI 时序统计基线与 frontier 行为下一步; 训练阶梯 (反思精化、反事实 DPO) 以本框架的可验证奖励为基础。代码与数据见 sky\_research 分支 090111mdiag。

## 参考文献

- [1] Fei et al. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. arXiv:2401.02051, 2024.
- [2] Ye et al. ReEvo: Large Language Models as Hyper-Heuristics with Reflection. NeurIPS, arXiv:2402.01145, 2024.
- [3] HeurAgenix: Evolving Heuristics via LLMs. arXiv:2506.15196, 2025.
- [4] CO-Bench: Benchmarking LLM Agents on Combinatorial Optimization. AAAI, arXiv:2504.04310, 2026.
- [5] ReflecSched: LLM Agents for Dynamic Flexible Job Shop Scheduling. arXiv:2508.01724, 2025.
- [6] DynaSchedBench: Dynamic Scheduling LLM Agent Benchmark. ICML, arXiv:2605.27566, 2026.
- [7] Chen et al. Automatic Root Cause Analysis via LLMs for Cloud Incidents. EuroSys, arXiv:2305.15778, 2023.
- [8] mABC: Multi-Agent Blockchain-Inspired Collaboration for Root Cause Analysis. EMNLP Findings, arXiv:2404.12135, 2024.
- [9] ITBench: Benchmarking Agentic AI for IT Operations. ICML, arXiv:2502.05352, 2025.
- [10] Cloud-OpsBench: Agentic Cloud Operations RCA Benchmark. arXiv:2603.00468, 2026.
- [11] OpenRCA 2.0: From Outcome Labels to Causal Process Supervision. arXiv:2606.27154, 2026.
- [12] LLM Agents with Digital Twins for Process Fault Handling. IEEE ETFA, arXiv:2505.02076, 2025.
- [13] LLM-Guided Heuristic Design from Simulation Traces. arXiv:2608.09343, 2026.
- [14] Wallat et al. Correctness is not Faithfulness in RAG Attributions. SIGIR, arXiv:2412.18004, 2024.
- [15] Klein et al. Noise-free comparison of stochastic agent-based simulations using common random numbers. arXiv:2409.02086, 2024.
- [16] CounterBench: Benchmarking Causal Counterfactual Reasoning in LLMs. arXiv:2502.11008, 2025.
- [17] Executable Counterfactuals: abduction steps in counterfactual benchmarks. arXiv:2510.01539, 2025.